

Word Embeddings from Scratch

Build a small skip-gram neural network and train embeddings for one Ethiopian language.

Use Python lists, loops, math and random. Implement the model, gradients and SGD yourself, without pretrained embeddings, automatic differentiation, NumPy or neural-network libraries. You may use the lesson code as a starting point.

1. Prepare text and training pairs

Choose an Ethiopian language and prepare a small text corpus. Apply suitable preprocessing, build a vocabulary and create center–context training pairs for your embedding model.

2. Implement the neural network

Use row-vector notation. Let V be vocabulary size, d embedding dimension, i the center word and t the context target. E has shape $V \times d$ and U has shape $d \times V$. Initialize both matrices with small random values using a recorded seed.

Operation	Required equation
Embedding lookup	$h = E[i]$ shape $1 \times d$
Scores	$s = hU$ shape $1 \times V$
Stable probabilities	$m = \max(s)$; $a[j] = \exp(s[j] - m)$; $p[j] = a[j] / \sum(a)$
One-pair loss	$L = (m - s[t]) + \ln(\sum(a))$
Score error	$e[j] = p[j] - 1$ if $j = t$, otherwise $e[j] = p[j]$
Gradients	$\text{grad_}U[k,j] = h[k] e[j]$; $\text{grad_}h[k] = \sum_j U[k,j] e[j]$
Update	$U \leftarrow U - \text{learning_rate} \times \text{grad_}U$; $E[i] \leftarrow E[i] - \text{learning_rate} \times \text{grad_}h$
Dataset loss	$J = (1/N) \sum_n L_n$, evaluated at the same fixed weights

Training implementation

Compute both gradients using the old E and U before updating either matrix. Update only the selected E row. Shuffle pairs each epoch.

Choose suitable embedding dimensions, context window, learning rate and number of epochs. Keep the model and corpus small enough to train locally.

3. Check the implementation

Run the first six checks below with tolerance 0.00001. The last two are optional. Debug with this tiny example before training on the local-language text.

Check	Input and expected result
Context window	Sentence [a,b,c,d], window 1: (a,b), (b,a), (b,c), (c,b), (c,d), (d,c). No cross-sentence pairs.
Forward pass	Vocabulary [cats,eat,food]; $E = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$; $U = \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & 1 \end{bmatrix}$; pair (cats,eat). Scores [0,1,-1].

Probabilities and loss	$p \approx [0.244728, 0.665241, 0.090031]$; $L \approx 0.407606$. Probabilities are nonnegative and sum to 1.
Gradients	$\text{grad}_U \approx [[0.244728, -0.334759, 0.090031], [0, 0, 0]]$; $\text{grad}_h \approx [-0.424790, 0.334759]$.
One update	Learning rate 0.1: $E[\text{cats}] \approx [1.042479, -0.033476]$. Recomputed loss ≈ 0.361859 ; other E rows unchanged.
Extreme scores	Scores $[1000, 0]$, target index 1: stable loss ≈ 1000 . Naive $-\ln(p_{\text{target}})$ may fail after underflow.
Shift invariance	Add 100 to every score: probabilities and loss remain equal within tolerance.
Saved model	Load the saved model: vocabulary, nearest-neighbor order and probabilities match the original within tolerance.

4. Train and query the embeddings

Train one model. Print mean training loss before training and after each epoch, using the same pairs and evaluating without changing weights.

Implement $\text{cosine}(a, b) = \text{sum}_k(a[k] \times b[k]) / (\text{sqrt}(\text{sum}_k a[k]^2) \times \text{sqrt}(\text{sum}_k b[k]^2))$.

For five vocabulary words, print the three nearest vectors from E , excluding the query itself. Handle unknown words and zero-norm vectors without crashing. A tiny corpus may produce weak neighbors; lower training loss alone does not establish semantic quality.

Optional additions

Save and reload E , U and the vocabulary using JSON.

Check an analytic gradient numerically: $[L(\theta + \epsilon) - L(\theta - \epsilon)] / (2\epsilon)$, with $\epsilon = 0.00001$.

Change only the context window or embedding dimension and compare nearest neighbors.